

CAPÍTULO 2: ANÁLISIS DE LA SOLUCIÓN Y ESTADO DEL ARTE

2.1. Estado del Arte

2.1.1. Antecedentes del Proyecto EDDIE (Evolución Histórica)

Para comprender los fundamentos sobre los que se sustenta la presente propuesta de modernización, es necesario reconstruir la trayectoria histórica del sistema EDDIE (Entorno Digital y Dinámico de Interacción Escrita), desarrollado de forma incremental en el Departamento de Ingeniería Informática de la Universidad de Santiago de Chile bajo el alero del laboratorio InTeractiOn. El proyecto constituye un caso representativo de software científico que ha evolucionado orgánicamente a través de seis memorias de título sucesivas, cada una incorporando un nuevo módulo funcional sin que existiera, en ningún momento del proceso, una revisión sistemática de la arquitectura común subyacente.

El concepto original fue propuesto por Carrasco Madriaga (2017), quien implementó un entorno de realidad aumentada móvil orientado a enriquecer la lectura en papel mediante dispositivos portátiles. Contreras Velásquez (2019) evolucionó esta propuesta hacia una configuración no obstructiva de escritorio, sustituyendo el dispositivo móvil por un sistema fijo de cámara y proyector que elimina la necesidad de mediadores físicos manuales. A partir de esta base, el proyecto se expandió mediante la incorporación sucesiva de tres capacidades de soporte al lector: Montanares Mundaca (2019) integró un módulo de búsqueda web contextual activado por voz y gestos; Pezoa Riquelme (2020) desarrolló un mecanismo de consistencia bidireccional entre el documento físico y su versión digital en PDF; y Estay Vera (2020) incorporó dispositivos de rastreo ocular (eye-tracking) para el análisis del comportamiento atencional del usuario. Posteriormente, Santibáñez Espinoza (2020) realizó un rediseño de la interfaz gráfica bajo un enfoque centrado en el usuario, y finalmente Ibaceta Jaña (2023) aplicó la metodología MECIVA PS para diagnosticar cuantitativamente el estado del software, identificando 67 errores estructurales acumulados a lo largo de este proceso de crecimiento.

Trabajo Investigador	Aporte al proyecto EDDIE	Limitaciones / Deuda técnica generada
Carrasco Madriaga (2017)	Prototipo inicial de lectura aumentada en dispositivos móviles.	Dependencia de mediadores manuales (pantalla móvil).
Contreras Velásquez (2019)	Configuración no obstructiva de escritorio (cámara + proyector).	Acoplamiento directo con la interfaz WinForms en C#.
Montanares Mundaca (2019)	Módulo de búsqueda web contextual (Wikipedia, diccionarios).	Carga mediante reflexión con Assembly.LoadFile.

Pezoa Riquelme (2020)	Módulo de consistencia bidireccional físico-digital (PDF).	Incompatibilidades de sincronización y fallo por rutas fijas.
Estay Vera (2020)	Integración de rastreo ocular (eye-tracking).	Carga aislada por Proxy Sandbox en AppDomain subordinado.
Santibáñez Espinoza (2020)	Rediseño centrado en el usuario de la interfaz gráfica.	Mantenimiento de la capa monolítica subyacente.
Ibaceta Jaña (2023)	Diagnóstico cuantitativo de errores mediante metodología MECIVA PS.	Revelación de 67 errores estructurales por falta de validación en hardware real.

Tabla 2.1: Síntesis de la evolución histórica del sistema EDDIE a través de seis memorias de título sucesivas.

Fuente: elaboración propia a partir de Carrasco Madriaga (2017), Contreras Velásquez (2019), Montanares Mundaca (2019), Pezoa Riquelme (2020), Estay Vera (2020), Santibáñez Espinoza (2020) e Ibaceta Jaña (2023).

Este proceso incremental produjo un sistema monolítico de aproximadamente 45.000 líneas de código distribuidas en 26 proyectos .csproj, en el cual la falta de una arquitectura común provocó la coexistencia de tres mecanismos heterogéneos de carga por reflexión (Assembly.LoadFrom, AppDomain Proxy Sandbox y Assembly.LoadFile), origen directo de más de 30 de los 67 errores documentados por Ibaceta Jaña (2023).

2.1.1.1. Catálogo de funcionalidades y levantamiento de la usabilidad previa

En su estado actual, EDDIE ofrece cinco categorías funcionales heredadas directamente de las memorias descritas: procesamiento de imágenes y reconocimiento gestual (Contreras Velásquez, 2019), búsqueda web contextual (Montanares Mundaca, 2019), consistencia bidireccional entre el documento físico y su versión digital (Pezoa Riquelme, 2020), rastreo ocular (Estay Vera, 2020), y una capa de presentación en WinForms resultante del rediseño centrado en el usuario de Santibáñez Espinoza (2020). Cada una de estas funcionalidades fue incorporada mediante un mecanismo de integración propio del tesista responsable, sin que se estableciera en ningún caso un contrato de interfaz común entre ellas, lo cual explica la heterogeneidad de mecanismos de carga identificada posteriormente en el diagnóstico del sistema.

El levantamiento de usabilidad realizado por Ibaceta Jaña (2023) mediante la metodología MECIVA PS constituye el diagnóstico más completo disponible sobre el estado funcional de EDDIE previo a esta propuesta. Su evaluación identificó fallas tanto internas (relacionadas con la consistencia de los módulos entre sí) como externas, vinculadas a la interacción del sistema con el hardware físico y a la ausencia de documentación de uso e instalación. Entre los hallazgos más críticos se cuentan fallas no resueltas en la conexión con hardware, como la inestabilidad de la cámara digital (error E16) y la imposibilidad de detectar zonas de la proyección mediante el eyetracker (error E49), lo que compromete directamente la validez de EDDIE como instrumento de medición científica del comportamiento atencional. Este diagnóstico funcional constituye la línea base contra la cual se contrastaron los resultados de la reconstrucción propuesta en este trabajo.

2.1.2. Deuda Técnica y Deuda de Reproducibilidad en Software Científico

Para efectos de este trabajo, se entiende por software científico aquel desarrollado ad hoc para responder a una tarea de investigación específica (como la captura, orquestación de sensores o el análisis de datos experimentales), en contraposición al software de propósito general que un laboratorio pueda emplear como herramienta auxiliar. Bajo esta definición, tanto EDDIE como su antecedente arquitectónico Multimodal Observer (MO) constituyen software científico, mientras que el sistema operativo o el entorno de desarrollo empleados durante su construcción no lo son, dado que no fueron concebidos para resolver un problema de investigación particular.

La literatura sobre ingeniería de software aplicada a la investigación documenta un fenómeno recurrente: las decisiones técnicas localmente óptimas (aquellas tomadas por un investigador para resolver el problema inmediato de su propia tesis, dentro del plazo acotado que esta impone) tienden a producir un sistema globalmente inconsistente cuando se acumulan a través de múltiples autores sin una arquitectura común (Ernst et al., 2018). Este fenómeno, denominado deuda técnica en software de investigación, se distingue de la deuda técnica convencional en que su origen no es exclusivamente la premura comercial, sino la estructura misma de la producción académica, donde cada memoria de título introduce su propio módulo bajo presión de tiempo y sin incentivos institucionales para revisar la base común. Deshpande et al. (2024) documentan una dinámica similar en la evolución de la investigación computacional a gran escala, donde la ausencia de prácticas de ingeniería de software sostenidas en el tiempo compromete la capacidad de los sistemas para escalar más allá de su propósito original.

Estrechamente relacionado, aunque conceptualmente distinto, se encuentra el fenómeno de la deuda de reproducibilidad (Reproducibility Debt, RpD), definida por Hassan et al. (2025) como la acumulación de imprecisiones, documentación insuficiente y variaciones técnicas no controladas que impiden recrear con éxito un flujo de trabajo computacional o experimental, ya sea por el autor original o por terceros. En el caso de EDDIE, esta deuda se manifiesta concretamente en la imposibilidad de validar buena parte de los módulos desarrollados durante la crisis sanitaria de 2019-2021 frente a hardware físico real, dado que el trabajo remoto impidió el acceso a cámaras y eyetrackers específicos (Ibaceta Jaña, 2023); en consecuencia, el software heredado no puede garantizar que sus resultados sean reproducibles en el entorno físico para el cual fue diseñado.

Un patrón análogo ha sido documentado en plataformas de mayor escala, como los sistemas de bioinformática y biología computacional, donde la falta de coordinación entre grupos de desarrollo produce herramientas difíciles de integrar entre sí y una consecuente pérdida de capacidad colaborativa (Orozco Solano, 2017). La literatura converge en que la vía de mitigación recurrente frente a ambos tipos de deuda es la adopción de arquitecturas modulares con interfaces estandarizadas, que separan la lógica propia de la investigación de la infraestructura técnica subyacente, fortaleciendo así las capas de procesos y datos del sistema (Deshpande et al., 2024; Orozco Solano, 2017).

Al momento de decidir cómo intervenir un sistema que arrastra este tipo de deuda, la literatura sobre actualización de software distingue distintos niveles de intervención según el Versionado Semántico (SemVer): correcciones incrementales o parches, que resuelven

errores puntuales sin alterar la base existente (cambios de tipo Patch o Minor); evoluciones mayores que introducen cambios arquitectónicos con potencial incompatibilidad hacia versiones anteriores (cambios Major); y, en los casos de obsolescencia tecnológica más severa, la reimplementación completa, que consiste en construir una nueva versión del sistema desde cero utilizando nuevas tecnologías base (Preston-Werner, s.f.; Syal, 2023). Esta tipología resulta directamente aplicable al diagnóstico de EDDIE: dado que sus 67 errores estructurales no son fallas aisladas sino la consecuencia de tres arquitecturas de carga incompatibles entre sí (Ibaceta Jaña, 2023), una intervención de tipo Patch o Minor resultaría insuficiente para eliminar la deuda técnica de raíz, lo que justifica considerar una intervención de mayor alcance como la propuesta en este trabajo.

2.1.3. Lectura Aumentada y Coordinación Multimodal

La lectura constituye un proceso cognitivo que ha sido objeto de estudio constante frente a la masificación de los dispositivos digitales. La evidencia en psicología cognitiva indica que, pese a la digitalización, el material impreso conserva ventajas de retención de ideas que no han sido replicadas completamente por el formato digital (Saleh & Mashhur, 2015; Singer & Alexander, 2016, como se citó en Santibáñez Espinoza, 2020). Esta asimetría se explica, en parte, porque el papel permite una manipulación táctil y natural del contenido, mientras que el entorno digital ofrece capacidades de automatización (búsqueda, sincronización, anotación dinámica) que resultan inalcanzables en el formato tradicional (Guimbretiére, 2003). La denominada lectura aumentada surge como un intento de conciliar ambos mundos, superponiendo capacidades digitales sobre el soporte físico sin desplazarlo.

En este dominio, EDDIE se sitúa junto a un conjunto de referentes internacionales que exploran distintas combinaciones de interacción tangible y proyección digital. DigitalDesk fue pionero en introducir un esquema de cámara y proyector sobre un escritorio físico para lograr interacción mixta entre papel y contenido digital, aunque sin incorporar rastreo ocular ni mecanismos de consistencia bidireccional. EnhancedDesk avanzó sobre este esquema mediante cámaras infrarrojas para el seguimiento de dedos y la proyección de contenido interactivo, sin contemplar módulos de análisis atencional. AR-PaperSync, por su parte, se concentró en la sincronización de literatura científica en realidad aumentada mediante dispositivos móviles, mientras que Enlight propuso una lámpara de escritorio inteligente con proyector y reconocimiento óptico de caracteres (OCR), sin integrar telemetría de la mirada. Frente a estos antecedentes, EDDIE se distingue por integrar simultáneamente proyección sobre papel, consistencia bidireccional físico-digital y eye-tracking, una combinación que no se replica íntegramente en ninguno de los sistemas revisados.

Coordinar esta combinación de sensores heterogéneos, los cuales operan a frecuencias de muestreo distintas, como el eye-tracking de alta frecuencia frente al procesamiento de texto de baja frecuencia, exige un modelo de orquestación en el cual un controlador central coordine el flujo de datos mediante patrones de diseño como el adaptador (Adapter Pattern), que permite unificar interfaces de dispositivos originalmente incompatibles bajo un contrato común (Megargel et al., 2021).

Dentro del propio laboratorio InTeractiOn, EDDIE puede contrastarse con dos referentes directos. El primero corresponde a la familia de catálogos de lectura

automatizada del tipo NLite, que emparejan de forma automática un libro físico con su versión digital equivalente sin requerir que el usuario indique manualmente dicha correspondencia al inicio de cada sesión; esta lógica de reconocimiento automático de catálogo es precisamente la que la propuesta de modernización de EDDIE busca incorporar, sustituyendo el emparejamiento manual del PDF que exige la versión actual. El segundo referente es el propio Multimodal Observer (MO), desarrollado en el mismo laboratorio para la captura, visualización y análisis de datos multimodales provenientes de sensores como teclado, mouse y sensores fisiológicos (Gutiérrez Gálvez, 2016). Si bien MO no aborda el dominio de la lectura, comparte con EDDIE el desafío estructural de coordinar múltiples fuentes de datos heterogéneas bajo un núcleo común, lo que lo convierte en el antecedente arquitectónico más relevante para la presente propuesta, según se detalla en la sección siguiente.

2.1.4. Arquitecturas Modulares y Patrones de Integración

En el dominio de los science gateways (plataformas que integran instrumentos y flujos de trabajo científicos), la separación entre la lógica de investigación y la infraestructura técnica se implementa habitualmente mediante dos elementos: una capa de abstracción de hardware (Hardware Abstraction Layer, HAL) que uniforma la comunicación con dispositivos físicos heterogéneos, y un catálogo de plugins cargados dinámicamente a través de un único mecanismo, lo que permite aislar el fallo de un componente sin comprometer el resto del sistema (Glatard et al., 2017). Esta misma arquitectura de núcleo liviano con contratos de interfaz bien definidos es la que permite minimizar el tiempo dedicado a resolver problemas comunes de integración, concentrando el esfuerzo de desarrollo en los algoritmos propios de cada dominio de investigación (Bustio Martínez et al., 2013).

Para fundamentar esta elección arquitectónica es necesario precisar tres conceptos que se emplean de forma recurrente en la literatura de ingeniería de software. La modularidad es un principio de diseño que consiste en dividir un sistema en componentes relativamente independientes, cada uno encapsulando un conjunto específico de funcionalidades e interactuando con los demás mediante interfaces bien definidas; entre sus beneficios se cuentan la mejora en la mantenibilidad, dado que los cambios dentro de un módulo tienen impacto limitado sobre el resto del sistema, y una mayor escalabilidad, ya que los módulos pueden añadirse o reemplazarse con facilidad (Brategren, s.f.). La extensibilidad, por su parte, es la capacidad de un sistema para incorporar nuevas funcionalidades sin modificar el código fuente de su núcleo, lo que reduce los tiempos y costos de desarrollo a largo plazo y permite que tanto los desarrolladores originales como terceros amplíen las capacidades del sistema (Prismic, 2024; VPN Unlimited, s.f.). Finalmente, un plugin es un componente de software que añade una funcionalidad específica a una aplicación existente, operando como un módulo relativamente independiente que se integra al programa principal sin alterar su código base; su uso favorece la personalización, ya que usuarios o desarrolladores pueden seleccionar únicamente los plugins que necesitan (BasuMallick, 2024; IONOS, 2020).

La viabilidad de este patrón arquitectónico se encuentra ampliamente demostrada en software de uso general. OBS Studio, por ejemplo, expone una API en C (libobs) que permite a los plugins (empaquetados como bibliotecas dinámicas cargadas al inicio de la aplicación) registrar nuevas fuentes de captura, filtros o servicios de salida sin modificar el núcleo del programa (Bailey, 2026; OBS, 2026). Visual Studio Code, por su parte, construye

gran parte de sus características centrales como extensiones que interactúan con el editor a través de una API pública, permitiendo desde la personalización visual hasta la incorporación de soporte para nuevos lenguajes de programación (VS Code, 2026). Ambos casos ilustran que un núcleo liviano con un contrato de extensión bien definido permite que un sistema crezca de forma sostenida sin comprometer su estabilidad interna, exactamente el atributo que se busca introducir en EDDIE mediante el contrato PluginBase propuesto.

El antecedente empírico más directo para esta propuesta, sin embargo, proviene del propio laboratorio InTeractiOn: el proyecto Multimodal Observer (MO). Concebido originalmente por Gutiérrez Gálvez (2016) como un software modular, extensible y multiplataforma para el trabajo con datos multimodales, MO se organizó desde su origen en tres capas: un núcleo (Core) que provee las funcionalidades base y el sistema de gestión de plugins, una capa de Módulos con funcionalidades específicas (Captura, Visualización, Análisis, Comunicación, Actualización y Wizard), y una capa de Extensiones o Plugins, que llegó a sumar 34 componentes desarrollados por sucesivos tesisistas del laboratorio (Curifuta Poblete, 2017; Vilches Céspedes, 2017; Cáceres Rodríguez, 2018; Polanco Berrios, 2019; Díaz Sandoval, 2021). No obstante, la continua evolución del núcleo para dar soporte a estos nuevos módulos derivó, con el tiempo, en incompatibilidades entre el núcleo y una parte importante de sus plugins, dejando varios de ellos inoperativos en las versiones más recientes de la aplicación, lo que resulta ser una crisis de deuda técnica estructuralmente idéntica a la diagnosticada en EDDIE. La solución adoptada por Figueroa Contreras (2025) consistió en reimplementar por completo el núcleo de MO en Python, bajo una arquitectura de plugins desacoplada y expuesta mediante un servicio local con FastAPI, lo que permitió restablecer la compatibilidad entre el núcleo y el conjunto de extensiones existentes, y validar el sistema resultante con métricas objetivas de calidad de código, rendimiento y aceptación en el mismo contexto académico. Esta experiencia constituye el precedente empírico que sustenta la decisión de reconstruir el núcleo de EDDIE bajo un esquema equivalente.

2.1.5. Metodologías de Evaluación de Software Científico

La validación de un sistema modernizado exige metodologías capaces de evaluar tanto la calidad técnica del software como su desempeño funcional frente a los requisitos heredados del sistema original. Para el caso de EDDIE, este trabajo se apoya en la metodología MECIVA PS, desarrollada y aplicada por Ibaceta Jaña (2023) específicamente para la evaluación y validación de software científico, y que constituye el instrumento mediante el cual se diagnosticaron los 67 errores estructurales que sirven de línea base para esta propuesta. MECIVA PS evalúa dos dimensiones centrales: la validez funcional, entendida como el porcentaje de requisitos heredados que un módulo cumple correctamente, y la confiabilidad, que mide la consistencia de los resultados obtenidos ante ejecuciones repetidas de una misma prueba, expresada mediante el coeficiente de variación de las métricas de cada componente. A esto se suma una escala de aceptación, la que fue empleada previamente por Estay Vera (2020) en la validación del módulo de rastreo ocular, que clasifica los resultados desde "Completa" (85-100% de cumplimiento) hasta "No aceptada" (0-14%), permitiendo comparar de forma estandarizada el desempeño de módulos evaluados en distintos momentos del proyecto.

Un marco de evaluación conceptualmente análogo fue empleado en la validación de Multimodal Observer. Soto Piña (2021) propuso un marco de trabajo específico para MO centrado en dos dimensiones equivalentes (validez y confiabilidad), donde la validez se establece mediante la comparación de los datos capturados por el sistema contra una línea

de referencia validada (ground truth), y la confiabilidad se determina mediante pruebas estadísticas que cuantifican el grado de consistencia de las mediciones bajo condiciones constantes. Figueroa Contreras (2025) reutilizó este mismo marco para revalidar los plugins de captura de MO tras la reimplementación de su núcleo, sustituyendo únicamente la herramienta de referencia externa por una biblioteca integrada en las pruebas automatizadas, y obtuvo errores relativos de 0% respecto al ground truth utilizado, evidencia de que una migración arquitectónica de esta naturaleza puede preservar la validez funcional del sistema original.

Complementariamente, ambos proyectos incorporan métricas de calidad de código como criterio de evaluación de la solidez arquitectónica: cobertura de pruebas unitarias e integración mediante frameworks como pytest, con un umbral mínimo de 80% adoptado por Figueroa Contreras (2025) para MO y replicado como criterio de evaluación en la presente propuesta para EDDIE; y análisis estático de código mediante SonarQube, que evalúa de forma automatizada la seguridad, confiabilidad y mantenibilidad del software (SonarQube, 2026). La convergencia de estos criterios (evaluación funcional heredada mediante MECIVA PS, validez y confiabilidad de datos mediante comparación con ground truth, y calidad de código mediante métricas estáticas) constituye el marco metodológico integral bajo el cual se evaluará la reconstrucción del núcleo de EDDIE propuesta en este trabajo.

REFERENCIAS

- Bailey, L. (2026). OBS Studio API Documentation. <https://docs.obsproject.com/>
- BasuMallick, C. (2024). What Are Plugins? Working, Importance, and Best Practices | Spiceworks. <https://www.spiceworks.com/tech/tech-general/articles/what-are-plugins/>
- Brategren, K. (s.f.). Software modularity. <https://www.modularmanagement.com/blog/software-modularity>
- Bustio Martínez, L., Coma Peña, Y., & Talavera Bustamante, I. (2013). Arquitectura basada en plugins para el desarrollo de software científico. En XV Convención y Feria Internacional Informática. La Habana, Cuba.
- Cáceres Rodríguez, C. F. (2018). Desarrollo de un módulo para la interacción, control y monitoreo remoto en el software Multimodal Observer [Memoria de título, Universidad de Santiago de Chile].
- Carrasco Madriaga, J. P. (2017). Diseño e implementación de un sistema de realidad aumentada para el apoyo de la lectura en papel [Memoria de título, Universidad de Santiago de Chile].
- Contreras Velásquez, C. E. (2019). Desarrollo de un sistema para soportar lectura aumentada no obstrusiva [Memoria de título, Universidad de Santiago de Chile].
- Curifuta Poblete, G. A. (2017). Extensiones de MO para el análisis de datos de rastreo ocular y expresiones faciales [Memoria de título, Universidad de Santiago de Chile].
- Deshpande, D., Chhugani, K., Ramesh, T., Pellegrini, M., Shiffman, S., Abedalthagafi, M. S., Alqahtani, S., Ye, J., Liu, X. S., Leek, J. T., et al. (2024). The evolution of computational research in a data-centric world. *Cell*, 187(17), 4449–4457. <https://doi.org/10.1016/j.cell.2024.07.045>

- Díaz Sandoval, F. I. (2021). Rediseño de interfaz de usuario y modelo de interacción de Multimodal Observer [Memoria de título, Universidad de Santiago de Chile].
- Ernst, N. A., Awon, A. M., Hingmire, S., & Li, Z. S. (2018). The nature of technical debt in research software. *Journal of the ACM*, 37(4), Artículo 111.
- Estay Vera, J. S. (2020). Integración de hardware de eye-tracking al sistema EDDIE [Memoria de título, Universidad de Santiago de Chile].
- Figueroa Contreras, M. (2025). Arquitectura basada en plugins para la captura y análisis de datos multimodales: El caso de Multimodal Observer [Memoria de título, Universidad de Santiago de Chile].
- Glatard, T., Rousseau, M. E., Camarasu-Pop, S., Adalat, R., Beck, N., Das, S., Ferreira da Silva, R., Khalili-Mahani, N., Korkhov, V., Quirion, P. O., et al. (2017). Software architectures to integrate workflow engines in science gateways. *Future Generation Computer Systems*, 75, 239–255. <https://doi.org/10.1016/j.future.2017.01.005>
- Guimbretière, F. (2003). Paper augmented digital documents. *Proceedings of the 16th Annual ACM Symposium on User Interface Software and Technology*.
- Gutiérrez Gálvez, C. J. (2016). MO: un software modular y extensible para la captura y visualización de datos multimodales [Memoria de título, Universidad de Santiago de Chile].
- Hassan, Z., Treude, C., Norrish, M., Williams, G., & Potanin, A. (2025). Characterising reproducibility debt in scientific software: A systematic literature review. *The Journal of Systems & Software*, 222, 112327. <https://doi.org/10.1016/j.jss.2024.112327>
- Ibaceta Jaña, I. A. (2023). Evaluación y validación de software científico mediante metodología MECIVA PS: Caso de estudio EDDIE [Memoria de título, Universidad de Santiago de Chile].
- IONOS. (2020). ¿Qué es un plugin y para qué se usa? <https://www.ionos.com/es-us/digitalguide/servidores/know-how/que-es-un-plugin/>
- Megargel, A., Shankararaman, V., & Walker, D. K. (2021). Referencia citada en trabajos previos del laboratorio InTeractiOn respecto al patrón adaptador; se recomienda verificar y completar el detalle bibliográfico exacto (título, fuente y editorial) antes de la entrega final.
- Montanares Mundaca, D. C. (2019). Desarrollo de un módulo de búsqueda web para el software EDDIE [Memoria de título, Universidad de Santiago de Chile].
- OBS. (2026). Plugins Guide. <https://obsproject.com/kb/plugins-guide>
- Orozco Solano, A. (2017). Diseño de Sistemas Bioinformáticos y Biología Computacional aplicados en Nanobiotecnología Molecular Médica [Tesis doctoral, Universidad Autónoma de Madrid].
- Pezoa Riquelme, I. E. (2020). Implementación de un módulo de consistencia bidireccional físico-digital para el sistema EDDIE [Memoria de título, Universidad de Santiago de Chile].

- Polanco Berrios, D. I. (2019). Desarrollo de un módulo para asistentes destinados a la gestión y despliegue de protocolos de investigación en MO [Memoria de título, Universidad de Santiago de Chile].
- Preston-Werner, T. (s.f.). Versionado Semántico 2.0.0. <https://semver.org/lang/es/>
- Prismic. (2024). What is extensibility? <https://prismic.io/blog/what-is-extensibility>
- Santibáñez Espinoza, C. A. (2020). Rediseño de interfaz de usuario para el sistema EDDIE bajo un enfoque centrado en el usuario [Memoria de título, Universidad de Santiago de Chile].
- SonarQube. (2026). Understanding measures and metrics | SonarQube Server Documentation. <https://docs.sonarsource.com/sonarqube-server/latest/user-guide/code-metrics/metrics-definition/>
- Soto Piña, M. A. (2021). Marco de trabajo para la evaluación de validez y confiabilidad de Multimodal Observer (MO) [Memoria de título, Universidad de Santiago de Chile].
- Syal, S. (2023). Demystifying Software Versioning: Understanding Patch, Minor, and Major Versions. <https://medium.com/@shilpasyal55/demystifying-software-versioning-understanding-patch-minor-and-major-versions-11927673f3d3>
- Vilches Céspedes, F. C. (2017). Módulo para la búsqueda, descarga, instalación y actualización de extensiones de MO [Memoria de título, Universidad de Santiago de Chile].
- VPN Unlimited. (s.f.). ¿Qué es la extensibilidad? – Términos y definiciones de ciberseguridad. <https://www.vpnunlimited.com/es/help/cybersecurity/extensibility>
- VS Code. (2026). Extension API. <https://code.visualstudio.com/api>